# Shared-memory Parallel Programming with OpenCilk


**Milind Chabbi**

**Nikhil Hegde**

**Department of Computer Science**
**IIT Dharwad**

# Outline for Today

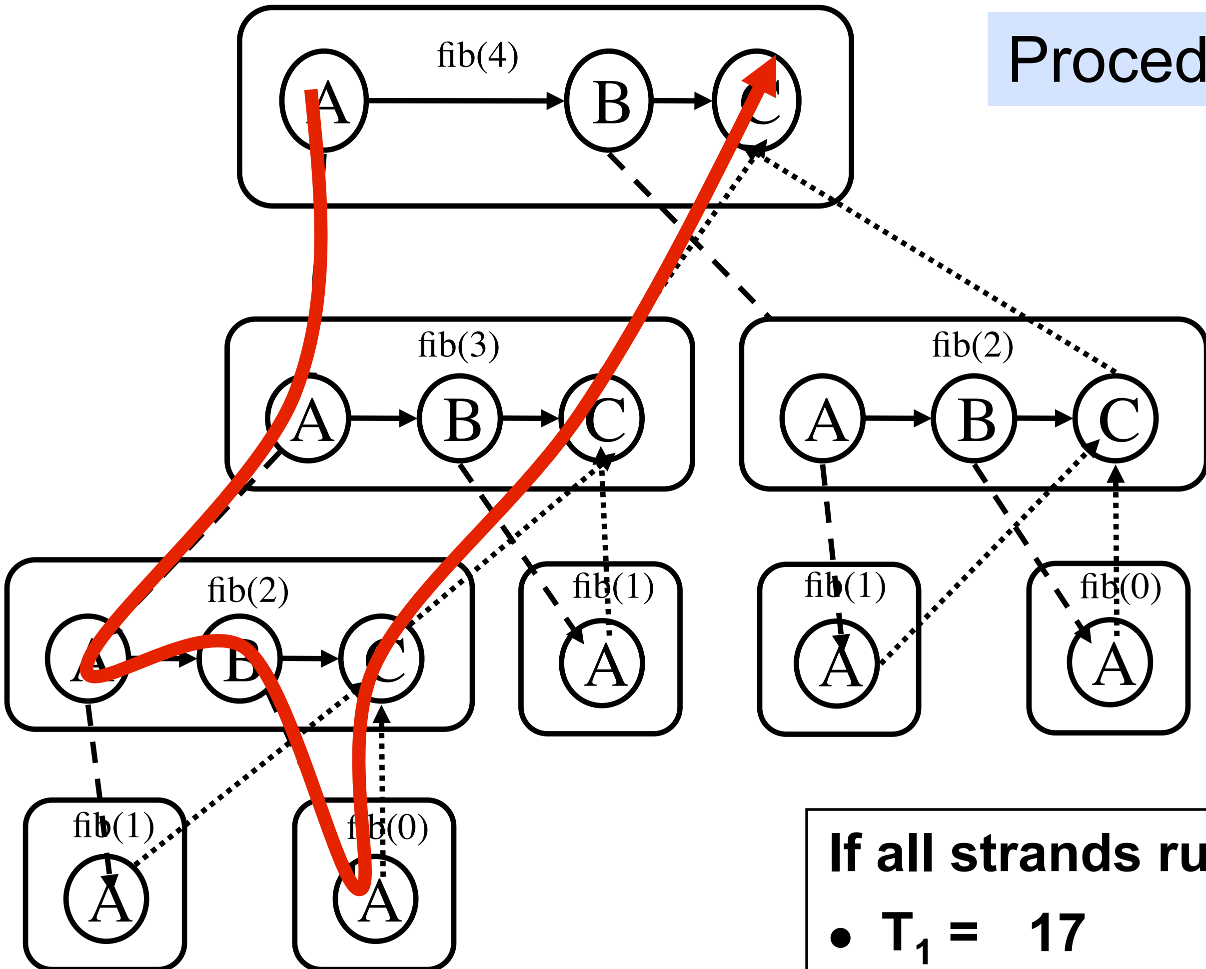
- **Assessing Cilk performance with CilkScale**

# Performance Measures

- $T_s$ = serial execution time
- $T_1$ = execution time on 1 processor (total work), $T_1 \geq T_s$
- $T_p$ = execution time on P processors
- $T_\infty$ = execution time on infinite number of processors
  - longest path in DAG
    - length reflects the cost of computation at nodes along the path
  - known as "critical path length"

# Work and Critical Path Example

Procedure oriented view




**If all strands run in unit time**

- **$T_1$ = 17**
- **$T_\infty$ = 8 (critical path length)**

# CilkScale - Scalability Analysis Tool

- Collects stats of parallel performance
- Measures work, span, parallelism
- Automatically benchmarks program in different number of processors
- Produce tables and graphical plots with the above performance and scalability measurements
- Predicts speedup on various numbers of processors based on work and span

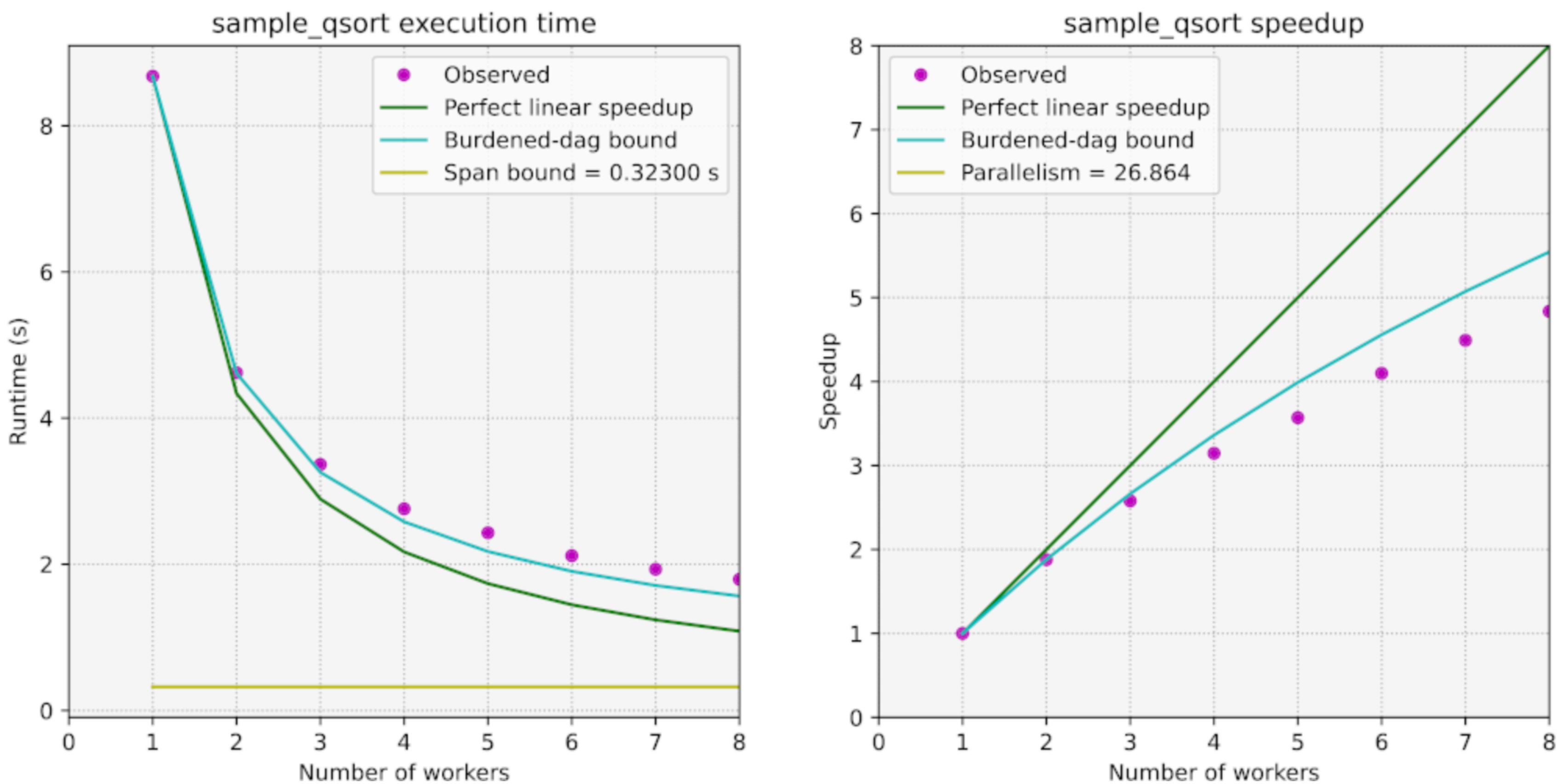
# CilkScale - Scalability Analysis Tool

- **Instruments Cilk program to collect performance measurements during its execution**
- **Cilkscale instrumentation operates in one of two modes**
  - ***Benchmarking* mode: Cilkscale measures the wall-clock execution time of your program**
    - **compile with -fopencilk -fcilktool=cilkscale-benchmark**
  - ***Work/span analysis* mode: Cilkscale measures the *work*, *span*, and *parallelism* of your program**
    - **compile with -fopencilk -fcilktool=cilkscale**
- **By default, Cilkscale only reports performance results for whole-program execution**
  - **fine-grained analyses of specific sub-computations is possible**

# Automatic benchmarks and visualization

ref: https://www.opencilk.org/doc/users-guide/cilkscale/

```
$ python3 /opt/opencilk/share/Cilkscale_vis/cilkscale.py \
    -c ./qsort_cs -b ./qsort_cs_bench \
    -ocsv cstable_qsort.csv -oplot csplots_qsort.pdf \
    --args 100000000
```



# Rule of Thumb

As a rule of thumb, the parallelism of a computation is deemed sufficient if it is about 10x larger (or more) than the number of available processors. On the other hand, if the parallelism is too high — say, several orders of magnitude higher than the number of processors — then the overhead for spawning tasks that are too fine-grained may become substantial

**Burdened Dag Bound:**

Slowed execution if every continuation incurs a steal (incurs overhead of parallelism)

# CilkScale Demo

- **Explore CilkScale for performance analysis using fib example**
  - `cilkscale_run.sh fib 47`
  - `cilkscale_run.sh fib_cutoff 47 2`
  - `cilkscale_run.sh fib_cutoff 47 5`
  - `cilkscale_run.sh fib_cutoff 47 10`
  - `cilkscale_run.sh fib_cutoff 47 20`

# References - I


- **Matteo Frigo, Charles Leiserson, and Keith Randall. The implementation of the Cilk-5 multithreaded language. In PLDI (Montreal, Quebec, Canada, June 17 - 19, 1998), 212-223.**
- **Mingdong Feng and Charles E. Leiserson. 1997. Efficient detection of determinacy races in Cilk programs. In *Proceedings of the ninth annual ACM symposium on Parallel algorithms and architectures* (SPAA '97). ACM, New York, NY, USA, 1-11.**
- **Guang-Ien Cheng, Mingdong Feng, Charles E. Leiserson, Keith H. Randall, and Andrew F. Stark. 1998. Detecting data races in Cilk programs that use locks. In *Proceedings of the tenth annual ACM symposium on Parallel algorithms and architectures* (SPAA '98). ACM, New York, NY, USA, 298-309.**

# References - II


- Yuxiong He, Charles E. Leiserson, and William M. Leiserson. 2010. The Cilkview scalability analyzer. In Proc. of the 22nd annual ACM symposium on Parallelism in algorithms and architectures (SPAA '10). ACM, New York, NY.
- Charles E. Leiserson. Cilk LECTURE 1. Supercomputing Technologies Research Group. Computer Science and Artificial Intelligence Laboratory. http://bit.ly/mit-cilk-lec1
- Matteo Frigo, Pablo Halpern, Charles E. Leiserson, and Stephen Lewin-Berlin. Reducers and other Cilk++ hyperobjects. SPAA '09, 79-90. Talk Slides. April 11, 2009. http://bit.ly/reducers
- Charles Leiserson, Bradley Kuzmaul, Michael Bender, and Hua-wen Jing. MIT 6.895 lecture notes - Theory of Parallel Systems. http://bit.ly/mit-6895-fall03
- Intel Cilk++ Programmer's Guide. Document # 322581-001US.